Module 1

Logistic Regression using Student Performance Dataset

Problem Statement

The objective of this project is to predict whether a student will pass or fail based on study hours, attendance, and previous marks using the Logistic Regression algorithm.

Dataset

Student Performance Dataset

Target Variable: Pass

- 1 = Pass
- 0 = Fail

Step 2: Import Libraries

In [8]:
```
import pandas as pd
import numpy as np

import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler

from sklearn.linear_model import LogisticRegression
```

```
from sklearn.metrics import accuracy_score
from sklearn.metrics import precision_score
from sklearn.metrics import recall_score
from sklearn.metrics import f1_score
from sklearn.metrics import confusion_matrix
from sklearn.metrics import classification_report
from sklearn.metrics import roc_curve
from sklearn.metrics import roc_auc_score
```

Step 3: Load Dataset

In [11]:

```
import pandas as pd

data = {
    "Hours_Studied": [2,4,6,8,3,5,7,1,2,5,6,7,8,4,3,9,2,6,5,7,8,1,4,6,7,3,5,8,9,2,4,6,7,5,3,8,9,1,2,6,7,4,5,8,3,6,7,9

    "Attendance": [60,70,85,95,65,80,90,55,58,78,82,88,96,72,68,98,62,84,79,87,93,57,74,81,89,66,76,91,99,61,73,83,86

    "Previous_Marks": [45,55,70,88,50,65,82,40,48,63,72,78,90,58,52,94,47,75,67,80,89,42,60,74,81,54,66,87,92,46,59,7

    "Pass": [0,0,1,1,0,1,1,0,0,1,1,1,1,0,0,1,0,1,1,1,1,0,0,1,1,0,1,1,0,0,1,1,0,1,1,0,0,1,1,1,0,1,1,0,0,1,1,0,1,1,0,1,1,1,0,1]
}

df = pd.DataFrame(data)

df
```

Out[11]:

	Hours_Studied	Attendance	Previous_Marks	Pass
0	2	60	45	0
1	4	70	55	0
2	6	85	70	1
3	8	95	88	1
4	3	65	50	0
5	5	80	65	1
6	7	90	82	1
7	1	55	40	0
8	2	58	48	0
9	5	78	63	1
10	6	82	72	1
11	7	88	78	1
12	8	96	90	1
13	4	72	58	0
14	3	68	52	0
15	9	98	94	1
16	2	62	47	0
17	6	84	75	1
18	5	79	67	1
19	7	87	80	1
20	8	93	89	1
21	1	57	42	0

	Hours_Studied	Attendance	Previous_Marks	Pass
22	4	74	60	0
23	6	81	74	1
24	7	89	81	1
25	3	66	54	0
26	5	76	66	1
27	8	91	87	1
28	9	99	92	1
29	2	61	46	0
30	4	73	59	0
31	6	83	73	1
32	7	86	79	1
33	5	77	68	1
34	3	69	53	0
35	8	94	91	1
36	9	97	95	1
37	1	54	39	0
38	2	59	44	0
39	6	85	76	1
40	7	88	83	1
41	4	71	57	0
42	5	75	64	1
43	8	92	86	1

	Hours_Studied	Attendance	Previous_Marks	Pass
44	3	67	51	0
45	6	82	71	1
46	7	90	84	1
47	9	100	96	1
48	2	63	49	0
49	5	78	69	1

In [ ]:
```
df = pd.read_csv("student.csv")
df.head()
```

Step 4: Exploratory Data Analysis (EDA)

Dataset Shape

In [ ]:
```
print(df.shape)
```

First Five Rows

In [ ]:
```
df.head()
```

Dataset Information

In [ ]:
```
df.info()
```

Missing Values

In [ ]:
```
df.isnull().sum()
```

Statistical Summary

In [ ]:
```
df.describe()
```

Pass / Fail Count

In [ ]:
```
sns.countplot(x="Pass", data=df)
plt.show()
```

Correlation

In [ ]:
```
plt.figure(figsize=(6,5))
sns.heatmap(df.corr(), annot=True, cmap="Blues")
plt.show()
```

Step 5: Data Preprocessing

In [ ]:
```
X = df.drop("Pass", axis=1)

y = df["Pass"]
```

Feature Scaling

In [ ]:
```
scaler = StandardScaler()

X = scaler.fit_transform(X)
```

Train Test Split

In [ ]:
```
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42
)
```

Step 6: Model Building

In [ ]:
```
model = LogisticRegression()
```

Step 7: Model Training

In [ ]:
```
model.fit(X_train, y_train)
```

Step 8: Prediction

In [ ]:
```
y_pred = model.predict(X_test)
```

Accuracy

In [ ]:
```
accuracy = accuracy_score(y_test, y_pred)

print("Accuracy =", accuracy)
```

Precision

In [ ]:
```
precision = precision_score(y_test, y_pred)

print("Precision =", precision)
```

Recall

In [ ]:
```
recall = recall_score(y_test, y_pred)

print("Recall =", recall)
```

F1 Score

In [ ]:
```
f1 = f1_score(y_test, y_pred)

print("F1 Score =", f1)
```

Confusion Matrix

In [ ]:
```
cm = confusion_matrix(y_test, y_pred)

sns.heatmap(cm, annot=True, fmt='d', cmap="Greens")

plt.xlabel("Predicted")

plt.ylabel("Actual")

plt.show()
```

Classification Report

In [ ]:
```
print(classification_report(y_test, y_pred))
```

ROC Curve

In [ ]:
```
y_prob = model.predict_proba(X_test)[:,1]

fpr, tpr, threshold = roc_curve(y_test, y_prob)

plt.plot(fpr, tpr)

plt.plot([0,1],[0,1],'r--')

plt.xlabel("False Positive Rate")

plt.ylabel("True Positive Rate")

plt.title("ROC Curve")

plt.show()
```

AUC Score

In [ ]:
```
auc = roc_auc_score(y_test, y_prob)

print("AUC Score =", auc)
```

Step 9: Hyperparameter Tuning

In [ ]:
```
from sklearn.model_selection import GridSearchCV

parameters = {
    "C":[0.01,0.1,1,10],
    "solver":["lbfgs","liblinear"]
}

grid = GridSearchCV(LogisticRegression(), parameters, cv=5)

grid.fit(X_train, y_train)

print(grid.best_params_)
```

Step 10: Result Interpretation

Result Interpretation

The Logistic Regression model successfully predicted whether a student passed or failed.

The model achieved high Accuracy, Precision, Recall, and F1 Score.

The ROC curve and AUC score indicate good classification performance.

Step 11: Conclusion

Conclusion

Logistic Regression is an effective algorithm for binary classification problems.

In this project, the model accurately predicted student pass/fail status based on study hours, attendance, and previous marks.

2. Problem Statement

Decision Tree Classifier

Problem Statement

The objective of this project is to predict whether a student will pass or fail based on study hours, attendance, previous marks, and assignments using the Decision Tree Classifier.

Dataset: Student Performance Dataset

2. Import Libraries

In [ ]:
```
import pandas as pd
import numpy as np

import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler

from sklearn.tree import DecisionTreeClassifier
from sklearn.tree import plot_tree

from sklearn.metrics import accuracy_score
from sklearn.metrics import precision_score
from sklearn.metrics import recall_score
from sklearn.metrics import f1_score
from sklearn.metrics import confusion_matrix
from sklearn.metrics import classification_report
```

3. Dataset

In [ ]:
```
import pandas as pd

data = {
    "Hours_Studied": [2,4,6,8,3,5,7,1,2,5,6,7,8,4,3,9,2,6,5,7,8,1,4,6,7,3,5,8,9,2,4,6,7,5,3,8,9,1,2,6,7,4,5,8,3,6,7,9
    "Attendance": [60,70,85,95,65,80,90,55,58,78,82,88,96,72,68,98,62,84,79,87,93,57,74,81,89,66,76,91,99,61,73,83,86
    "Previous_Marks": [45,55,70,88,50,65,82,40,48,63,72,78,90,58,52,94,47,75,67,80,89,42,60,74,81,54,66,87,92,46,59,7
    "Pass": [0,0,1,1,0,1,1,0,0,1,1,1,1,0,0,1,0,1,1,1,1,0,0,1,1,0,1,1,1,0,0,1,1,1,0,1,1,0,0,1,1,0,1,1,0,1,1,1,0,1]
}

df = pd.DataFrame(data)

df.head()
```

4. EDA

In [ ]:
```
print(df.shape)

df.info()

df.describe()

df.isnull().sum()
```

In [ ]:
```
sns.countplot(x="Pass", data=df)
plt.show()
```

In [ ]:
```
sns.heatmap(df.corr(), annot=True, cmap="coolwarm")
plt.show()
```

5. Data Preprocessing

In [ ]:
```
X = df.drop("Pass", axis=1)

y = df["Pass"]
```

In [ ]:
```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
```

```
)
```

6. Model Building

In [ ]:
```
model = DecisionTreeClassifier(random_state=42)
```

7. Model Training

In [ ]:
```
model.fit(X_train, y_train)
```

8. Prediction

In [ ]:
```
y_pred = model.predict(X_test)
```

9. Evaluation

In [ ]:
```
print("Accuracy:", accuracy_score(y_test, y_pred))
print("Precision:", precision_score(y_test, y_pred))
print("Recall:", recall_score(y_test, y_pred))
print("F1 Score:", f1_score(y_test, y_pred))
```

In [ ]:
```
print(classification_report(y_test, y_pred))
```

In [ ]:
```
cm = confusion_matrix(y_test, y_pred)

sns.heatmap(cm, annot=True, fmt="d", cmap="Blues")

plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.title("Confusion Matrix")
plt.show()
```

10. Decision Tree Visualization

In [ ]:
```
plt.figure(figsize=(15,8))

plot_tree(
    model,
    feature_names=X.columns,
    class_names=["Fail","Pass"],
    filled=True
)

plt.show()
```

11. Hyperparameter Tuning

In [ ]:
```
from sklearn.model_selection import GridSearchCV

params = {
    "max_depth": [2, 3, 4, 5, 6],
    "criterion": ["gini", "entropy"]
}

grid = GridSearchCV(
    DecisionTreeClassifier(random_state=42),
    params,
    cv=5
)

grid.fit(X_train, y_train)

print("Best Parameters:", grid.best_params_)
```

12. Result Interpretation

The Decision Tree Classifier successfully predicted student pass/fail status.

The model learned decision rules based on study hours, attendance, and previous marks. The confusion matrix and evaluation metrics indicate good classification performance.

13. Conclusion

Decision Tree is an easy-to-understand classification algorithm. It provides interpretable decision rules and performed well on the student performance dataset.

3. Problem Statement

Random Forest Classifier

Problem Statement

The objective of this project is to predict whether a student will pass or fail based on study hours, attendance, previous marks, and assignments using the Random Forest Classifier.

Dataset: Student Performance Dataset

2. Import Libraries

In [13]:
```
import pandas as pd
import numpy as np

import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score
from sklearn.metrics import confusion_matrix, classification_report
```

3. Dataset

In [ ]:
```
import pandas as pd

data = {
    "Hours_Studied": [2,4,6,8,3,5,7,1,2,5,6,7,8,4,3,9,2,6,5,7,8,1,4,6,7,3,5,8,9,2,4,6,7,5,3,8,9,1,2,6,7,4,5,8,3,6,7,9
    "Attendance": [60,70,85,95,65,80,90,55,58,78,82,88,96,72,68,98,62,84,79,87,93,57,74,81,89,66,76,91,99,61,73,83,86
    "Previous_Marks": [45,55,70,88,50,65,82,40,48,63,72,78,90,58,52,94,47,75,67,80,89,42,60,74,81,54,66,87,92,46,59,7
    "Pass": [0,0,1,1,0,1,1,0,0,1,1,1,1,0,0,1,0,1,1,1,1,0,0,1,1,0,1,1,1,0,0,1,1,1,0,1,1,0,0,1,1,0,1,1,0,1,1,1,0,1]
}

df = pd.DataFrame(data)

df.head()
```

4. EDA

In [ ]:
```
print(df.shape)
df.head()
df.info()
df.describe()
df.isnull().sum()
```

In [ ]:
```
sns.countplot(x="Pass", data=df)
plt.show()
```

In [ ]:
```
sns.heatmap(df.corr(), annot=True, cmap="coolwarm")
plt.show()
```

5. Data Preprocessing

In [ ]:
```
X = df.drop("Pass", axis=1)
y = df["Pass"]

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
```

6. Model Building

In [ ]:
```
model = RandomForestClassifier(
    n_estimators=100,
    random_state=42
)
```

7. Model Training

In [ ]:
```
model.fit(X_train, y_train)
```

8. Prediction

In [ ]:
```
y_pred = model.predict(X_test)
```

9. Evaluation

In [ ]:
```
print("Accuracy:", accuracy_score(y_test, y_pred))
print("Precision:", precision_score(y_test, y_pred))
print("Recall:", recall_score(y_test, y_pred))
print("F1 Score:", f1_score(y_test, y_pred))
```

In [ ]:
```
print(classification_report(y_test, y_pred))
```

In [ ]:
```
cm = confusion_matrix(y_test, y_pred)

sns.heatmap(cm, annot=True, fmt="d", cmap="Greens")

plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.title("Confusion Matrix")
plt.show()
```

Feature Importance

In [ ]:
```
importance = pd.DataFrame({
    "Feature": X.columns,
    "Importance": model.feature_importances_
})

importance = importance.sort_values(
    by="Importance",
    ascending=False
)

plt.figure(figsize=(8,5))

sns.barplot(
    x="Importance",
    y="Feature",
    data=importance
)

plt.title("Feature Importance")
plt.show()
```

10. Hyperparameter Tuning

In [ ]:
```
from sklearn.model_selection import GridSearchCV

params = {
    "n_estimators": [50, 100, 150],
    "max_depth": [3, 5, 7]
}

grid = GridSearchCV(
    RandomForestClassifier(random_state=42),
    params,
    cv=5
)
```

```
grid.fit(X_train, y_train)

print(grid.best_params_)
```

11. Result Interpretation

The Random Forest Classifier accurately predicted whether students passed or failed. It combined multiple decision trees to improve prediction accuracy and reduce overfitting.

12. Conclusion

Random Forest achieved excellent classification performance. Compared with a single Decision Tree, it provided better accuracy and more reliable predictions.

4. Problem Statement

Support Vector Machine (SVM)

Problem Statement

The objective of this project is to predict whether a student will pass or fail based on study hours, attendance, and previous marks using the Support Vector Machine (SVM) algorithm.

Dataset

Student Performance Dataset

2. Import Libraries

In [ ]:
```
import pandas as pd
import numpy as np
```

```
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.svm import SVC

from sklearn.metrics import accuracy_score
from sklearn.metrics import precision_score
from sklearn.metrics import recall_score
from sklearn.metrics import f1_score
from sklearn.metrics import confusion_matrix
from sklearn.metrics import classification_report
from sklearn.metrics import roc_curve
from sklearn.metrics import roc_auc_score
```

3. Dataset

In [ ]:
```
import pandas as pd

data = {
    "Hours_Studied": [2,4,6,8,3,5,7,1,2,5,6,7,8,4,3,9,2,6,5,7,8,1,4,6,7,3,5,8,9,2,4,6,7,5,3,8,9,1,2,6,7,4,5,8,3,6,7,9

    "Attendance": [60,70,85,95,65,80,90,55,58,78,82,88,96,72,68,98,62,84,79,87,93,57,74,81,89,66,76,91,99,61,73,83,86

    "Previous_Marks": [45,55,70,88,50,65,82,40,48,63,72,78,90,58,52,94,47,75,67,80,89,42,60,74,81,54,66,87,92,46,59,7

    "Pass": [0,0,1,1,0,1,1,0,0,1,1,1,1,0,0,1,0,1,1,1,1,0,0,1,1,0,1,1,1,0,0,1,1,1,0,1,1,0,0,1,1,0,1,1,0,1,1,1,0,1]
}

df = pd.DataFrame(data)

df
```

4. EDA

In [ ]:
```
print(df.shape)

df.head()

df.info()

df.describe()

df.isnull().sum()
```

Target Distribution

In [ ]:
```
sns.countplot(x="Pass", data=df)
plt.show()
```

Correlation Heatmap

In [ ]:
```
plt.figure(figsize=(6,5))
sns.heatmap(df.corr(), annot=True, cmap="coolwarm")
plt.show()
```

5. Data Preprocessing

In [ ]:
```
X = df.drop("Pass", axis=1)

y = df["Pass"]
```

Feature Scaling

In [ ]:
```
scaler = StandardScaler()

X = scaler.fit_transform(X)
```

Train-Test Split

In [ ]:
```
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
```

```
    test_size=0.2,
    random_state=42
)
```

6. Model Building

In [ ]:
```
model = SVC(
    kernel="rbf",
    probability=True,
    random_state=42
)
```

7. Model Training

In [ ]:
```
model.fit(X_train, y_train)
```

8. Prediction

In [ ]:
```
y_pred = model.predict(X_test)
```

9. Evaluation

In [ ]:
```
print("Accuracy:", accuracy_score(y_test, y_pred))
print("Precision:", precision_score(y_test, y_pred))
print("Recall:", recall_score(y_test, y_pred))
print("F1 Score:", f1_score(y_test, y_pred))
```

In [ ]:
```
print(classification_report(y_test, y_pred))
```

Confusion Matrix

In [ ]:
```
cm = confusion_matrix(y_test, y_pred)

sns.heatmap(cm, annot=True, fmt="d", cmap="Blues")
```

```
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.title("Confusion Matrix")

plt.show()
```

ROC Curve

In [ ]:
```
y_prob = model.predict_proba(X_test)[:,1]

fpr, tpr, threshold = roc_curve(y_test, y_prob)

plt.plot(fpr, tpr, label="ROC Curve")
plt.plot([0,1],[0,1],"r--")

plt.xlabel("False Positive Rate")
plt.ylabel("True Positive Rate")
plt.title("ROC Curve")
plt.legend()

plt.show()
```

AUC Score

In [ ]:
```
print("AUC Score:", roc_auc_score(y_test, y_prob))
```

10. Hyperparameter Tuning

In [ ]:
```
from sklearn.model_selection import GridSearchCV

params = {
    "C": [0.1, 1, 10],
    "kernel": ["linear", "rbf"]
}

grid = GridSearchCV(
    SVC(probability=True),
    params,
```

```
    cv=5
)

grid.fit(X_train, y_train)

print("Best Parameters:", grid.best_params_)
```

11. Result Interpretation

Result Interpretation

The Support Vector Machine model successfully classified students as pass or fail. Feature scaling improved the model performance, and the ROC-AUC score indicates good classification capability.

12. Conclusion

Conclusion

Support Vector Machine is an effective classification algorithm. It achieved good accuracy in predicting student performance and handled the binary classification task efficiently.

5. Problem Statement

K-Nearest Neighbors (KNN)

Problem Statement

The objective of this project is to predict whether a student will pass or fail based on study hours, attendance, previous marks, and assignments using the K-Nearest Neighbors (KNN) algorithm.

Dataset: Student Performance Dataset

2. Import Libraries

In [ ]:
```
import pandas as pd
import numpy as np

import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler

from sklearn.neighbors import KNeighborsClassifier

from sklearn.metrics import accuracy_score
from sklearn.metrics import precision_score
from sklearn.metrics import recall_score
from sklearn.metrics import f1_score
from sklearn.metrics import confusion_matrix
from sklearn.metrics import classification_report
from sklearn.metrics import roc_curve
from sklearn.metrics import roc_auc_score
```

3. Dataset

In [ ]:
```
import pandas as pd

data = {
    "Hours_Studied": [2,4,6,8,3,5,7,1,2,5,6,7,8,4,3,9,2,6,5,7,8,1,4,6,7,3,5,8,9,2,4,6,7,5,3,8,9,1,2,6,7,4,5,8,3,6,7,9

    "Attendance": [60,70,85,95,65,80,90,55,58,78,82,88,96,72,68,98,62,84,79,87,93,57,74,81,89,66,76,91,99,61,73,83,86

    "Previous_Marks": [45,55,70,88,50,65,82,40,48,63,72,78,90,58,52,94,47,75,67,80,89,42,60,74,81,54,66,87,92,46,59,7

    "Pass": [0,0,1,1,0,1,1,0,0,1,1,1,1,0,0,1,0,1,1,1,1,0,0,1,1,0,1,1,0,0,1,1,0,1,1,0,1,1,1,0,1,1,0,0,1,1,0,1,1,0,1,1,1,0,1]
}
```

```
df = pd.DataFrame(data)

df
```

4. Exploratory Data Analysis (EDA)

In [ ]:
```
print("Dataset Shape:", df.shape)

df.head()

df.info()

df.describe()

df.isnull().sum()
```

Pass / Fail Distribution

In [ ]:
```
sns.countplot(x="Pass", data=df)
plt.title("Pass and Fail Distribution")
plt.show()
```

Correlation Heatmap

In [ ]:
```
plt.figure(figsize=(6,5))

sns.heatmap(df.corr(), annot=True, cmap="coolwarm")

plt.title("Correlation Matrix")

plt.show()
```

5. Data Preprocessing

In [ ]:
```
X = df.drop("Pass", axis=1)

y = df["Pass"]
```

Feature Scaling

In [ ]:
```
scaler = StandardScaler()

X = scaler.fit_transform(X)
```

Train-Test Split

In [ ]:
```
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42
)
```

6. Model Building

In [ ]:
```
model = KNeighborsClassifier(n_neighbors=5)
```

7. Model Training

In [ ]:
```
model.fit(X_train, y_train)
```

8. Prediction

In [ ]:
```
y_pred = model.predict(X_test)
```

9. Model Evaluation

Accuracy

In [ ]:
```
print("Accuracy:", accuracy_score(y_test, y_pred))
```

Precision

In [ ]:
```
print("Precision:", precision_score(y_test, y_pred))
```

Recall

In [ ]:
```
print("Recall:", recall_score(y_test, y_pred))
```

F1 Score

In [ ]:
```
print("F1 Score:", f1_score(y_test, y_pred))
```

Classification Report

In [ ]:
```
print(classification_report(y_test, y_pred))
```

Confusion Matrix

In [ ]:
```
cm = confusion_matrix(y_test, y_pred)

sns.heatmap(cm, annot=True, fmt="d", cmap="Blues")

plt.xlabel("Predicted")

plt.ylabel("Actual")

plt.title("Confusion Matrix")

plt.show()
```

ROC Curve

In [ ]:
```
y_prob = model.predict_proba(X_test)[:,1]

fpr, tpr, threshold = roc_curve(y_test, y_prob)

plt.plot(fpr, tpr, label="ROC Curve")
```

```
plt.plot([0,1],[0,1],"r--")

plt.xlabel("False Positive Rate")

plt.ylabel("True Positive Rate")

plt.title("ROC Curve")

plt.legend()

plt.show()
```

AUC Score

In [ ]:
```
print("AUC Score:", roc_auc_score(y_test, y_prob))
```

10. Hyperparameter Tuning

In [ ]:
```
from sklearn.model_selection import GridSearchCV

parameters = {
    "n_neighbors": [3,5,7,9],
    "weights": ["uniform","distance"]
}

grid = GridSearchCV(
    KNeighborsClassifier(),
    parameters,
    cv=5
)

grid.fit(X_train, y_train)

print("Best Parameters:", grid.best_params_)
print("Best Score:", grid.best_score_)
```

11. Result Interpretation

The K-Nearest Neighbors (KNN) model successfully classified students as pass or fail based on the nearest data points. The model achieved good accuracy, precision, recall, and F1-score, demonstrating effective performance on the student performance dataset.

12. Conclusion

The K-Nearest Neighbors algorithm is a simple and effective classification technique. It accurately predicted student pass/fail status using study hours, attendance, and previous marks, making it suitable for this binary classification problem.

6. Problem Statement

Naive Bayes Classifier

The objective of this project is to predict whether a student will pass or fail based on study hours, attendance, previous marks, and assignments using the Naive Bayes Classifier.

Dataset

Student Performance Dataset

2. Import Libraries

In [ ]:
```
import pandas as pd
import numpy as np

import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import GaussianNB

from sklearn.metrics import accuracy_score
from sklearn.metrics import precision_score
```

```
from sklearn.metrics import recall_score
from sklearn.metrics import f1_score
from sklearn.metrics import confusion_matrix
from sklearn.metrics import classification_report
from sklearn.metrics import roc_curve
from sklearn.metrics import roc_auc_score
```

3. Dataset

In [ ]:
```
import pandas as pd

data = {
    "Hours_Studied": [2,4,6,8,3,5,7,1,2,5,6,7,8,4,3,9,2,6,5,7,8,1,4,6,7,3,5,8,9,2,4,6,7,5,3,8,9,1,2,6,7,4,5,8,3,6,7,9

    "Attendance": [60,70,85,95,65,80,90,55,58,78,82,88,96,72,68,98,62,84,79,87,93,57,74,81,89,66,76,91,99,61,73,83,86

    "Previous_Marks": [45,55,70,88,50,65,82,40,48,63,72,78,90,58,52,94,47,75,67,80,89,42,60,74,81,54,66,87,92,46,59,7

    "Pass": [0,0,1,1,0,1,1,0,0,1,1,1,1,0,0,1,0,1,1,1,1,0,0,1,1,0,1,1,0,0,1,1,0,1,1,0,1,1,1,0,1,1,0,0,1,1,0,1,1,0,1,1,1,0,1]
}

df = pd.DataFrame(data)

df
```

4. Exploratory Data Analysis (EDA)

In [ ]:
```
print("Dataset Shape:", df.shape)

df.head()

df.info()

df.describe()

df.isnull().sum()
```

Pass/Fail Distribution

In [ ]:
```
sns.countplot(x="Pass", data=df)

plt.title("Pass and Fail Distribution")

plt.show()
```

Correlation Heatmap

In [ ]:
```
plt.figure(figsize=(6,5))

sns.heatmap(df.corr(), annot=True, cmap="viridis")

plt.title("Correlation Matrix")

plt.show()
```

5. Data Preprocessing

In [ ]:
```
X = df.drop("Pass", axis=1)

y = df["Pass"]
```

Train-Test Split

In [ ]:
```
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42
)
```

6. Model Building

In [ ]:
```
model = GaussianNB()
```

7. Model Training

In [ ]:
```
model.fit(X_train, y_train)
```

8. Prediction

In [ ]:
```
y_pred = model.predict(X_test)
```

9. Model Evaluation

Accuracy

In [ ]:
```
print("Accuracy:", accuracy_score(y_test, y_pred))
```

Precision

In [ ]:
```
print("Precision:", precision_score(y_test, y_pred))
```

Recall

In [ ]:
```
print("Recall:", recall_score(y_test, y_pred))
```

F1 Score

In [ ]:
```
print("F1 Score:", f1_score(y_test, y_pred))
```

Classification Report

In [ ]:
```
print(classification_report(y_test, y_pred))
```

Confusion Matrix

In [ ]:
```
cm = confusion_matrix(y_test, y_pred)
```

```
sns.heatmap(cm, annot=True, fmt="d", cmap="Oranges")

plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.title("Confusion Matrix")

plt.show()
```

ROC Curve

In [ ]:
```
y_prob = model.predict_proba(X_test)[:,1]

fpr, tpr, threshold = roc_curve(y_test, y_prob)

plt.plot(fpr, tpr, label="ROC Curve")

plt.plot([0,1],[0,1],"r--")

plt.xlabel("False Positive Rate")
plt.ylabel("True Positive Rate")

plt.title("ROC Curve")

plt.legend()

plt.show()
```

AUC Score

In [ ]:
```
print("AUC Score:", roc_auc_score(y_test, y_prob))
```

10. Hyperparameter Tuning

In [ ]:
```
print(model.get_params())
```

11. Result Interpretation

The Gaussian Naive Bayes model successfully classified students into pass and fail categories. It produced good accuracy and classification metrics with fast execution time.

12. Conclusion

Gaussian Naive Bayes is a simple and efficient classification algorithm. It performed well on the student performance dataset and provided quick predictions with satisfactory accuracy.

7. Problem Statement

Gradient Boosting Classifier

The objective of this project is to predict whether a student will pass or fail based on study hours, attendance, previous marks, and assignments using the Gradient Boosting Classifier.

Dataset

Student Performance Dataset

2. Import Libraries

In [ ]:
```
import pandas as pd
import numpy as np

import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.ensemble import GradientBoostingClassifier

from sklearn.metrics import accuracy_score
from sklearn.metrics import precision_score
```

```
from sklearn.metrics import recall_score
from sklearn.metrics import f1_score
from sklearn.metrics import confusion_matrix
from sklearn.metrics import classification_report
from sklearn.metrics import roc_curve
from sklearn.metrics import roc_auc_score
```

3. Dataset

In [ ]:
```
import pandas as pd

data = {
    "Hours_Studied": [2,4,6,8,3,5,7,1,2,5,6,7,8,4,3,9,2,6,5,7,8,1,4,6,7,3,5,8,9,2,4,6,7,5,3,8,9,1,2,6,7,4,5,8,3,6,7,9

    "Attendance": [60,70,85,95,65,80,90,55,58,78,82,88,96,72,68,98,62,84,79,87,93,57,74,81,89,66,76,91,99,61,73,83,86

    "Previous_Marks": [45,55,70,88,50,65,82,40,48,63,72,78,90,58,52,94,47,75,67,80,89,42,60,74,81,54,66,87,92,46,59,7

    "Pass": [0,0,1,1,0,1,1,0,0,1,1,1,1,0,0,1,0,1,1,1,1,0,0,1,1,0,1,1,0,0,1,1,0,1,1,0,1,1,1,0,1,1,0,0,1,1,0,1,1,0,1,1,1,0,1]
}

df = pd.DataFrame(data)

df
```

4. Exploratory Data Analysis (EDA)

In [ ]:
```
print(df.shape)

df.head()

df.info()

df.describe()

df.isnull().sum()
```

Target Distribution

In [ ]:
```
sns.countplot(x="Pass", data=df)
plt.title("Pass and Fail Distribution")
plt.show()
```

Correlation Heatmap

In [ ]:
```
plt.figure(figsize=(6,5))

sns.heatmap(df.corr(), annot=True, cmap="coolwarm")

plt.title("Correlation Matrix")

plt.show()
```

5. Data Preprocessing

In [ ]:
```
X = df.drop("Pass", axis=1)

y = df["Pass"]
```

Train-Test Split

In [ ]:
```
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42
)
```

6. Model Building

In [ ]:
```
model = GradientBoostingClassifier(
    random_state=42
)
```

7. Model Training

In [ ]:
```
model.fit(X_train, y_train)
```

8. Prediction

In [ ]:
```
y_pred = model.predict(X_test)
```

9. Model Evaluation

In [ ]:
```
# Accuracy
print("Accuracy:", accuracy_score(y_test, y_pred))
```

Precision

In [ ]:
```
print("Precision:", precision_score(y_test, y_pred))
```

Recall

In [ ]:
```
print("Recall:", recall_score(y_test, y_pred))
```

F1 Score

In [ ]:
```
print("F1 Score:", f1_score(y_test, y_pred))
```

Classification Report

In [ ]:
```
print(classification_report(y_test, y_pred))
```

Confusion Matrix

In [ ]:
```
cm = confusion_matrix(y_test, y_pred)

sns.heatmap(cm, annot=True, fmt="d", cmap="Greens")
```

```
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.title("Confusion Matrix")

plt.show()
```

ROC Curve

In [ ]:
```
y_prob = model.predict_proba(X_test)[:,1]

fpr, tpr, threshold = roc_curve(y_test, y_prob)

plt.plot(fpr, tpr, label="ROC Curve")

plt.plot([0,1],[0,1],"r--")

plt.xlabel("False Positive Rate")
plt.ylabel("True Positive Rate")

plt.title("ROC Curve")

plt.legend()

plt.show()
```

AUC Score

In [ ]:
```
print("AUC Score:", roc_auc_score(y_test, y_prob))
```

10. Hyperparameter Tuning

In [ ]:
```
from sklearn.model_selection import GridSearchCV

parameters = {
    "n_estimators":[50,100,150],
    "learning_rate":[0.01,0.1,0.2],
    "max_depth":[2,3]
}
```

```
grid = GridSearchCV(
    GradientBoostingClassifier(random_state=42),
    parameters,
    cv=5
)

grid.fit(X_train,y_train)

print("Best Parameters:",grid.best_params_)
```

11. Result Interpretation

The Gradient Boosting Classifier achieved high classification performance by combining multiple weak learners. The model produced strong accuracy, precision, recall, and F1-score while reducing prediction errors.

12. Conclusion

Gradient Boosting is a powerful ensemble learning algorithm that provides highly accurate predictions. It successfully classified students as pass or fail and performed better than many basic classification models.